

ExMe: Keep Improving With Extrapolation and Merging

Yiguan Lin , Bin Xu, Yinghao Li, and Yang Gao 

Abstract—Model merging, a typical parameter editing technique, combines different models in expectation of superior performance, without relying on the substantial computational resources and high-quality annotated data that instruction tuning heavily depends on. However, this approach lacks well-defined optimization objectives during parameter integration, making consistent performance gains in the merged model unattainable. This paper attempts to provide a clear optimization direction for model merging. We first validate the effectiveness of model extrapolation during the instruction fine-tuning phase, and then propose the Extrapolation Merging paradigm, which continuously improves model performance without requiring additional computational resources or data. Through the extrapolation method, we provide a clear direction for model merging and achieve local optimization region search, thereby ensuring that the merged model exhibits significant performance improvements. We conduct experiments on multiple different tasks, and the results show that our method consistently and stably enhances model performance after fine-tuning. Our main contributions are: (1) validating the effectiveness of model extrapolation between the base model and the supervised fine-tuned model; (2) revealing that extrapolation has a performance boundary where moderate coefficients improve performance but excessive ones cause degradation; and (3) proposing ExMe, which combines interpolation and extrapolation to achieve stable performance gains.

Index Terms—Large language model, model merging, model extrapolation.

I. INTRODUCTION

LARGE language models (LLMs) typically require an alignment phase to effectively understand and follow human instructions [1], [2], [3], [4]. This phase often involves supervised post-training, such as fine-tuning (SFT) and reinforcement learning from human feedback (RLHF) [5], [6], both of which demand high-quality data for downstream tasks and significant computational resources [7]. In contrast, model editing offers a more efficient alternative by enabling direct modification of a

model's knowledge or behavior after initial training, eliminating the need for full retraining [8], [9], [10], [11], [12], [13]. Unlike alignment techniques that demand extensive data and computational resources, these methods allow for targeted updates while preserving the model's core capabilities, with model merging and model extrapolation being two representative approaches.

Some existing works in *Model Merging* research demonstrate that model performance can be significantly enhanced through weight merging techniques, eliminating the need for additional training data or substantial computational resources [14], [15]. This breakthrough presents a promising alternative paradigm for model alignment that could potentially overcome traditional limitations. However, current model merging methodologies predominantly focus on hyperparameter optimization and redundant parameter identification [16], [17], often resulting in merged models whose performance merely interpolates between the constituent models [18]. These strategies fail to consider that model parameters inherently carry optimization directions, thereby introducing uncertainty into the performance of the merged model. Hence, providing a clear direction for model merging is a critical issue.

Model Extrapolation (EXPO) [19] is a method derived from model merging that similarly enhances model performance by adjusting parameter matrix without requiring additional data or training. By employing an inverse strategy to model merging, model extrapolation views DPO/RLHF [20] models as the outcome of merging an SFT model with an unknown but more powerful model (EXPO Model), thereby enabling the inference of EXPO model's parameters. Therefore, extrapolation can be readily employed to obtain models with better overall performance. However, how to further utilize extrapolation models remains a problem that needs to be addressed.

Nevertheless, model extrapolation unidirectionally amplifies the optimization trajectory established during the SFT phase, without accounting for the multiple potential optimization paths that a model may develop during training. If only a model checkpoint from a single optimization path is selected for model merging, the model is prone to getting stuck in a local optimum region. Furthermore, since the DPO/RLHF training phase takes the SFT model as its starting point, the model parameters after this phase are already in a relatively optimal position, leading to a narrower optimization direction for DPO/RLHF on this model. Contrastly, if extrapolation is employed based on the base model during the SFT phase and multiple optimization paths are considered, it would provide a broader optimization space for the

Received 25 December 2025; revised 7 May 2026; accepted 27 June 2026. Date of publication 30 June 2026; date of current version 8 August 2026. This work supported in part by the Major Research Plan of the National Natural Science Foundation of China under Grant 92370110 and in part by the National Natural Science Foundation of China General Program under Grant 62576038. Recommended for acceptance by X. Liu. (Yiguan Lin and Bin Xu contributed equally to this work.) (Corresponding author: Yang Gao.)

Yiguan Lin, Bin Xu, and Yang Gao are with the School of Computer Science, Technology, Beijing Institute of Technology, Beijing 100811, China (e-mail: yglin@bit.edu.cn; binxu@bit.edu.cn; gyang@bit.edu.cn).

Yinghao Li is with the School of Computer, Information Technology, Shanxi University, Taiyuan 030006, China (e-mail: yh@sxu.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3708509

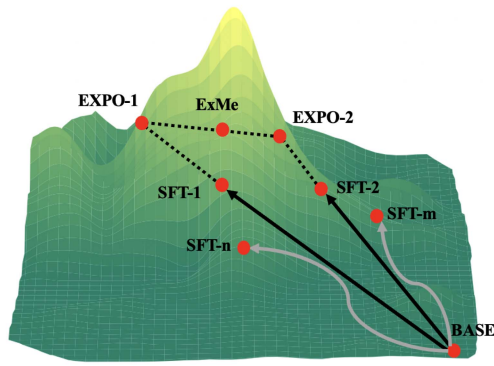


Fig. 1. Overview of ExMe: This figure provides an abstract conceptual illustration of the ExMe pipeline rather than real-data experimental results. The ExMe process begins with a base model, which is first fine-tuned using high-quality supervised data to steer the optimization of model parameters. After evaluation, the two best-performing checkpoints are selected. These fine-tuned models are then independently extrapolated, further amplifying the advantages gained in the previous phase, resulting in two extrapolated models: EXPO-1 and EXPO-2. Finally, the strengths of both extrapolated models are combined through a weighted merging operation, yielding the final merged model. In this schematic diagram, model performance is represented by color shading—lighter colors indicate stronger performance.

model and enable model merging based on these optimization paths, thereby stabilizing the achievement of a model with better performance.

In this paper, we propose a paradigm for model alignment that can be achieved without additional data or computational power, called **Extrapolation Merging (ExMe)**, which integrates model extrapolation and merging methods. This approach enhances model performance comprehensively by adjusting model parameters alone. As illustrated in Fig. 1, by performing extrapolation on the SFT model, we obtain multiple models that are optimized along different directions, and then merge these models. Specifically, we begin by fine-tuning the base model to obtain multiple SFT models, which are considered the result of merging the base model with an unknown but more powerful model. By applying the extrapolation method during the SFT stage, we obtain models with superior performance, thereby validating the effectiveness of the extrapolation approach on the SFT stage. Furthermore, after determining multiple optimization directions using the extrapolation method, we select the two best-performing models as the starting points for model merging. This approach yields a merged model with higher comprehensive performance across different optimization directions. Our contributions are as follows:

- We demonstrate the effectiveness of model extrapolation between base and SFT models, validating its utility in the supervised fine-tuning stage.
- We establish a performance boundary for extrapolation, showing that moderate coefficients yield improvements while excessive values lead to degradation—providing a clear and effective optimization direction for model merging.
- We propose ExMe, a simple yet effective paradigm that combines interpolation and extrapolation to achieve stable

performance gains without requiring additional computational resources, training, or fine-tuning data.

The structure of this paper is as follows: First, it systematically reviews relevant research on model merging and model extrapolation. It then proposes a bidirectional optimization paradigm, providing a complete derivation of the basic merging formula, the variant equations for extrapolation, and a unified mathematical framework, along with theoretical proofs. Subsequently, multi-dimensional evaluation experiments are conducted using Qwen3 series models of varying scales on both general instruction-tuning datasets and math-related datasets. The experimental results show that: (1) extrapolation methods outperform traditional merging but exhibit a performance ceiling; (2) performance collapse occurs when the extrapolation coefficient exceeds a certain threshold; and (3) the proposed ExMe framework not only surpasses the performance limits of single extrapolation models but also achieves a synergistic optimization of stability and performance.

II. RELATED WORK

A. Fine-Tuning of LLMs

Fine-tuning is a critical step in adapting large models to downstream tasks. After pre-training on vast amounts of unlabeled text, large models acquire general language understanding capabilities but often lack the specificity required for solving particular tasks. Fine-tuning addresses this by further training the model on labeled datasets containing task-specific input-output pairs, enhancing its ability to generate accurate and relevant responses in applications such as question answering, summarization, and translation. From the perspective of training data and methodology, fine-tuning can be categorized into three technical approaches: supervised fine-tuning (SFT), reinforcement learning fine-tuning based on human feedback (RLHF), and reinforcement learning fine-tuning based on AI feedback (RLAIF). In terms of parameter scale, fine-tuning can be divided into two paths: full fine-tuning (FFT), which updates all model parameters, and parameter-efficient fine-tuning (PEFT), which updates only a subset of parameters. Balancing cost and performance, PEFT has emerged as the mainstream solution.

Among PEFT methods, Prefix Tuning [21] prepends trainable prefix embeddings to the input layer to steer model outputs and improve training efficiency. Adapter Tuning inserts small neural networks (adapters) between layers, keeping the original parameters frozen and training only the adapters. Prompt Tuning [22] optimizes "soft prompts" to condition frozen language models for specific downstream tasks. LoRA [23] observed that when adapting models to specific tasks, the parameter matrices are often over-parametrized—despite their high dimensionality, they exhibit a low intrinsic rank. To address this, LoRA proposes adding low-rank decomposition matrices to the pre-trained model's parameter matrices to approximate the updates for each layer, significantly reducing the number of parameters that need to be trained. QLoRA further refines LoRA by incorporating high-precision weights and learnable low-rank adapters, leveraging quantization and low-rank matrix decomposition to further reduce fine-tuning costs.

Compared to full-parameter fine-tuning, these PEFT methods maintain model performance while substantially lowering training costs, making them widely adopted in parameter-efficient fine-tuning of open-source large language models.

B. Model Merging and Model Extrapolation

Model merging has emerged as a research trend in recent years, aiming to leverage existing models with different capabilities to efficiently create a unified model with diverse capabilities by adjusting weight parameters.

Model averaging [24] is a common method for model merging, constructing the merged model by averaging the weight parameters of different models. Weighted merging assigns different weights to the models involved, with Fisher-merging employing the Fisher information matrix to determine the optimal merging hyperparameters [25]. Bayesian search [26] begins with a series of pre-trained checkpoints and uses Bayesian optimization to find the optimal merging hyperparameters. MoLE [27] approaches merging within a single model, treating each trained LoRA as an independent expert, controlling hierarchical weights by integrating a learnable gating function within each layer to learn the optimal weight combinations for different objectives. DARE [17] points out that the delta parameters of various SFT models (the disparity between fine-tuned and pre-trained parameters) are highly redundant, and thus employs DARE to eliminate redundant incremental parameters in each model before merging them. PCB-MERGING [28] enhances model merging stability by re-calibrating task vectors at the parameter level through parameter competition balancing. KnOTS [29] employs singular value decomposition (SVD) to map LoRA model parameters into a shared space with aligned elements, achieving parameter alignment during model fusion.

However, these model merging methods do not take into account the optimization direction of parameters. Model extrapolation, derived from weighted merging, adopts a reverse approach by treating the DPO/RLHF model as an interpolated result between an SFT model and a stronger model, thereby facilitating the extrapolation of a more powerful model from the SFT model and the existing DPO/RLHF model. Unlike traditional model merging methods, model extrapolation considers the optimization direction of parameters, using human preference data to guide parameter adjustments and linearly amplifying the effects of reinforcement learning on these adjustments. Model extrapolation has so far demonstrated effectiveness primarily between SFT models and DPO/RLHF models.

III. METHOD

In this section, we first introduce the mechanisms of model merging and extrapolation method. We then conclude by presenting our ExMe method.

A. Background

1) *Model Merging*: Model merging aims to utilize multiple distinct model checkpoints by applying mathematical operations, such as weighted merging, to merge them into a unified

model that consolidates the strengths of each. This technique is capable of efficiently enhancing model performance. Formally, let $\Theta_1, \Theta_2, \dots, \Theta_t$ represent t checkpoints with the same architecture. The linear combination of these model checkpoints in the parameter space can be expressed in the following form, which is a general weighted merging formula:

$$\Theta_{merged} = \sum_{i=1}^t \lambda_i \Theta_i \quad \text{s.t.} \quad \sum_{i=1}^t \lambda_i = 1 \quad (1)$$

where $\lambda_i \in [0, 1]$ represents the weight of the i -th model checkpoint involved in the merging process, and Θ_{merged} represents the merged model resulting from the merging process.

Merging more models does not necessarily improve the performance of the merged model. Previous studies have shown that when merging checkpoints from models with the same architecture, the performance of the merged model often declines as the number of models involved increases. Typically, merging two models yields the best results [26]. Therefore, this paper employs a strategy of merging only two checkpoints, and (1) can be rewritten as:

$$\Theta_{merged} = \lambda \Theta_1 + (1 - \lambda) \Theta_2 \quad (2)$$

where $\lambda \in [0, 1]$ represents the weight of Θ_1 , and $1 - \lambda$ represents the weight of Θ_2 in the merging process.

2) *Model Extrapolation*: Model extrapolation is a variant of model merging techniques that exploits parameter trends between a weaker and a stronger model to extrapolate a more powerful model.

Formally, let Θ_{weak} represents the weaker model and Θ_{strong} represents the stronger model. The stronger model Θ_{strong} can be seen as the result of merging Θ_{weak} with an unknown but more powerful model, denoted as Θ_{expo} , according to (2) that can be rewritten as:

$$\Theta_{strong} = \lambda \Theta_{expo} + (1 - \lambda) \Theta_{weak} \quad (3)$$

Then by substituting (3), we can obtain:

$$\Theta_{expo} = \frac{\Theta_{strong} - (1 - \lambda) \Theta_{weak}}{\lambda} \quad (4)$$

$$= \frac{1}{\lambda} \Theta_{strong} - \frac{1 - \lambda}{\lambda} \Theta_{weak} \quad (5)$$

Through the above (5), extrapolated models can be easily obtained. Essentially, extrapolation linearly amplifies the advantages gained from SFT, thereby surpassing the performance ceiling of SFT.

B. ExMe

Extrapolation Merging is a method that leverages extrapolation to push models further in the direction of their optimization trajectory, while using merging to integrate the complementary strengths of different extrapolated models.

ExMe begins by fine-tuning the base model to obtain a series of SFT models, denoted as Θ_{ckpt_1} to Θ_{ckpt_n} . Next, a comprehensive evaluation of the saved SFT models is conducted, and the two models with the best capabilities are selected—in

this paper, we consider two scenarios separately: comprehensive capability across all tasks and individual mathematical capability—denoted as Θ_{sft_1} and Θ_{sft_2} .

Next, by applying the model extrapolation on SFT stage, we treat the SFT model and base model as the aforementioned strong and weak model, respectively. According to (5), by setting $\alpha = \frac{1}{\lambda} - 1$, we can obtain several extrapolated models by manually setting different extrapolation hyperparameters. Then we select two extrapolated models, denoted as Θ_{expo_1} and Θ_{expo_2} , as shown in the following equations:

$$\Theta_{expo_1} = \Theta_{sft_1} + \alpha_1(\Theta_{sft_1} - \Theta_{base}) \quad (6a)$$

$$\Theta_{expo_2} = \Theta_{sft_2} + \alpha_2(\Theta_{sft_2} - \Theta_{base}) \quad (6b)$$

Finally, by adjusting various merging hyperparameter β , we merge Θ_{expo_1} and Θ_{expo_2} according to (2), leading to several candidate models, which is formulated as:

$$\Theta_{ExMe} = \beta\Theta_{expo_1} + (1 - \beta)\Theta_{expo_2} \quad (7)$$

where $\beta \in [0, 1]$ represents the weight of Θ_{expo_1} , and $1 - \beta$ represents the weight of Θ_{expo_2} in the merging process. After evaluating these candidate models, we select the best performing one as the final ExMe model.

C. Theoretical Explanation

Since we train on the same model for a limited number of steps, we can assume that the resulting multiple models all lie within the same low-loss connected region. This condition satisfies the requirements of Linear Mode Connectivity (LMC) [30], [31], [32]. Under this premise, the behavior of large models can be approximately described using the linearization method of the Neural Tangent Kernel (NTK) [33], [34].

a) Assumption 1: Starting from the same initial model Θ_{base} , we obtain two models through training: $\Theta_{sft_1} = \Theta_{base} + \tau_1$, which excels at solving task T_1 , where τ_1 denotes the task vector of T_1 ; and $\Theta_{sft_2} = \Theta_{base} + \tau_2$, which excels at solving task T_2 , where τ_2 denotes the task vector of T_2 . These two models, together with the original model Θ_{base} , reside in the same low-loss connected region [35]. Specifically, for any $\lambda \in (0, 1)$, the loss of the interpolated model $\lambda\Theta_{sft_1} + (1 - \lambda)\Theta_{sft_2}$ remains approximately equal to the losses of Θ_{sft_1} and Θ_{sft_2} , i.e.,

$$\begin{aligned} \mathcal{L}(\lambda\Theta_{sft_1} + (1 - \lambda)\Theta_{sft_2}) &\approx \mathcal{L}(\Theta_{sft_1}) \\ &\approx \mathcal{L}(\Theta_{sft_2}). \end{aligned} \quad (8)$$

b) Assumption 2: There exists an unknown optimal task vector τ^* such that the corresponding model $\Theta^* = \Theta_{base} + \tau^*$ achieves lower loss on the joint task $T_1 \cup T_2$ than Θ_{train_1} and Θ_{train_2} individually on the same joint task. In other words, τ^* represents an ideal combination of τ_1 and τ_2 . We further assume that τ^* lies in the cone spanned by τ_1 and τ_2 , i.e., there exist $a, b \geq 0$ such that $\tau^* \approx a\tau_1 + b\tau_2$.

c) Theorem: Under assumptions 1 and 2, there exist extrapolation coefficients $\alpha_1 > 0$, $\alpha_2 > 0$ and a merging weight $\beta \in (0, 1)$ such that the final model defined by

$$\Theta_{final} = \Theta_{base} + \beta\lambda_1\tau_1 + (1 - \beta)\lambda_2\tau_2 \quad (9)$$

satisfies

$$\mathcal{L}(\Theta_{final}) \leq \min\{\mathcal{L}(\Theta_{sft_1}), \mathcal{L}(\Theta_{sft_2})\}. \quad (10)$$

d) Step 1: First, starting from the base model Θ_{base} , fine-tuning on downstream tasks yields two models:

$$\Theta_{sft_1} = \Theta_{base} + \tau_1, \quad \Theta_{sft_2} = \Theta_{base} + \tau_2. \quad (11)$$

Next, with extrapolation coefficients $\alpha_1 > 0$, $\alpha_2 > 0$, we construct the extrapolated models:

$$\begin{cases} \Theta_{expo_1} = \Theta_{base} + (1 + \alpha_1)\tau_1, \\ \Theta_{expo_2} = \Theta_{base} + (1 + \alpha_2)\tau_2. \end{cases} \quad (12)$$

Let $\lambda_1 = 1 + \alpha_1 > 1$ and $\lambda_2 = 1 + \alpha_2 > 1$. Then,

$$\Theta_{expo_1} = \Theta_{base} + \lambda_1\tau_1, \quad \Theta_{expo_2} = \Theta_{base} + \lambda_2\tau_2. \quad (13)$$

Finally, we merge the two extrapolated models with a weighted average:

$$\Theta_{final} = \beta\Theta_{expo_1} + (1 - \beta)\Theta_{expo_2}. \quad (14)$$

Simplifying gives the parametric form of the final model:

$$\Theta_{final} = \Theta_{base} + \beta\lambda_1\tau_1 + (1 - \beta)\lambda_2\tau_2. \quad (15)$$

e) Step 2: Within the NTK linearization regime, the loss function with respect to the task vector can be approximated as a convex quadratic form. Let the optimal joint task vector be τ^* ; then for any task vector τ , the NTK squared distance to the optimal vector is given by

$$d^2(\tau, \tau^*) = |\tau - \tau^*|_H^2. \quad (16)$$

For a single task vector τ_1 , consider scaling it along its own direction: $\lambda\tau_1$. Then

$$d^2(\lambda\tau_1, \tau^*) = \lambda^2|\tau_1|_H^2 - 2\lambda\langle\tau_1, \tau^*\rangle_H + |\tau^*|_H^2. \quad (17)$$

This is a convex quadratic function in λ , whose unique minimizer is

$$\lambda_1^{\text{opt}} = \frac{\langle\tau_1, \tau^*\rangle_H}{|\tau_1|_H^2}. \quad (18)$$

A convex quadratic function decreases monotonically to the left of its minimizer. Therefore, if $\lambda_1^{\text{opt}} > 1$, then for any $\lambda_1 \in (1, \lambda_1^{\text{opt}}]$, we have

$$d^2(\lambda_1\tau_1, \tau^*) < d^2(\tau_1, \tau^*), \quad (19)$$

and hence $d(\lambda_1\tau_1, \tau^*) < d(\tau_1, \tau^*)$. In other words, there exists an extrapolation coefficient $\lambda_1 > 1$ such that the extrapolated task vector is closer to the optimal direction. Similarly, if $\lambda_2^{\text{opt}} > 1$, then there exists a corresponding $\lambda_2 > 1$.

f) Step 3: Let the extrapolated task vectors be $\tau'_1 = \lambda_1\tau_1$ and $\tau'_2 = \lambda_2\tau_2$. From Assumption 1 and the properties of the NTK regime, τ'_1 and τ'_2 still lie in the same low-loss connected region; therefore, any convex combination $\beta\tau'_1 + (1 - \beta)\tau'_2$ (with $\beta \in (0, 1)$) also resides in that region.

Define the function $g(\beta)$ as the squared distance between the convex combination and the optimal vector τ^* :

$$g(\beta) = \|\beta\tau'_1 + (1 - \beta)\tau'_2 - \tau^*\|_H^2. \quad (20)$$

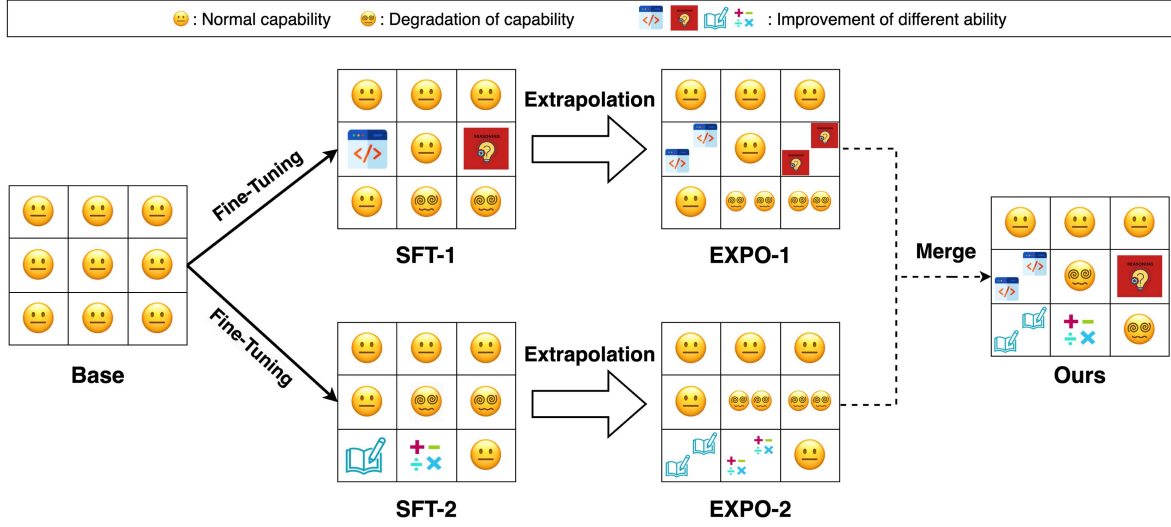


Fig. 2. Three stages of ExMe: In the fine-tuning stage, after the base model is fine-tuned, a series of checkpoints are generated, from which the two SFT models with the best overall performance are selected for the next stage of extrapolation. In the extrapolation stage, each of these SFT models emphasizes different capabilities, which are further amplified in the best two directions. In the merging stage, the merging process balances the strengths and mitigates the weaknesses of the two extrapolated models, resulting in improved overall performance.

Here, $g(\beta)$ is a convex quadratic function in β , attaining its minimum at

$$\beta^* = \frac{\langle \tau'_2 - \tau^*, \tau'_2 - \tau'_1 \rangle_H}{\|\tau'_2 - \tau'_1\|_H^2}. \quad (21)$$

Since τ'_1 and τ'_2 are already closer to τ^* than τ_1 and τ_2 respectively, and they point in different directions, the convex combination with the optimal weight β^* yields a distance to τ^* no greater than that of either τ'_1 or τ'_2 :

$$d(\beta^* \tau'_1 + (1 - \beta^*) \tau'_2, \tau^*) \leq \min(d(\tau'_1, \tau^*), d(\tau'_2, \tau^*)). \quad (22)$$

Equality holds only when $\tau'_1 = \tau'_2$ or when one of them already equals τ^* ; in general, the inequality is strict, meaning that two vectors from different directions, both closer to τ^* , complement each other through convex combination and thus further approach the optimal direction.

g) Step 4: In the NTK linearization regime, the loss function is Lipschitz continuous with respect to the model parameters [33]: there exists a constant $C > 0$ such that for any parameter increments Δ and Δ' ,

$$|\mathcal{L}(\Theta_{base} + \Delta) - \mathcal{L}(\Theta_{base} + \Delta')| \leq C \cdot \|\Delta - \Delta'\|_H. \quad (23)$$

Consequently, we have the following inequalities:

$$\begin{cases} \mathcal{L}(\Theta_{final}) - \mathcal{L}(\Theta_{base} + \tau^*) \leq C \cdot d(\beta^* \tau'_1 + (1 - \beta^*) \tau'_2, \tau^*), \\ \mathcal{L}(\Theta_{sft1}) - \mathcal{L}(\Theta_{base} + \tau^*) \leq C \cdot d(\tau_1, \tau^*). \end{cases} \quad (24)$$

Combining the distance relations from Steps 2 and 3:

$$d(\beta^* \tau'_1 + (1 - \beta^*) \tau'_2, \tau^*) \leq d(\tau'_1, \tau^*) < d(\tau_1, \tau^*), \quad (25)$$

We obtain that there exist extrapolation coefficients $\alpha_1 > 0$, $\alpha_2 > 0$ and a merging weight $\beta \in (0, 1)$ such that $\mathcal{L}(\Theta_{final}) \leq \mathcal{L}(\Theta_{sft1})$. By symmetry, $\mathcal{L}(\Theta_{final}) \leq \mathcal{L}(\Theta_{sft2})$ also holds. The theorem shows that, with appropriate extrapolation and merging hyperparameters, first extrapolating the two models individually and then merging them yields a final model whose

loss on the combined task is no higher than that of either original model. Equations (24) and (25) also indicate that this method is not universally effective; its performance depends crucially on the proper choice of the extrapolation and merging hyperparameters.

IV. EXPERIMENTS

A. Pipeline

We present the pipeline of the ExMe method in Fig. 2, which consists of three stages: Supervised Fine-Tuning, Extrapolation, and Merging.

SFT Stage: We first fine-tune the base model using an instruction-following dataset, then evaluate a series of SFT models and select the top two models based on their performance. As shown in Fig. 2, SFT-1 and SFT-2 represent the selected models.

Extrapolation Stage: In this stage, we use a model extrapolation method with manually specified coefficients $\alpha \in \{0.1, 0.2, \dots, 0.5\}$. Following (6a) and (6b), we extrapolate the selected SFT models, SFT-1 and SFT-2, away from the base model. The two extrapolated models as EXPO-1 and EXPO-2 are then selected for the model merging stage.

Merging Stage: Finally, based on (7), we merge the two selected extrapolated models with the manually set merging hyperparameter $\beta \in \{0.1, 0.2, \dots, 0.9\}$. Then we evaluate these merged models to select the one with the best performance as the final ExMe model.

B. Models

To comprehensively validate the effectiveness of the proposed method, we conduct experiments on multiple models of different scales, including 1.7B, 4B, and 8B parameter sizes. We select the following representative models as our base models: Qwen3-1.7B, Qwen3-4B, Qwen3-8B [36]. All employed models are original base versions rather than instruction-tuned versions.

TABLE I

THIS TABLE SHOWS THE SCORES OF QWEN3-1.7B, QWEN3-4B, AND QWEN3-8B USING DIFFERENT METHODS, WITH SFT MODELS TRAINED ON THE ALPACA DATASET. * INDICATES THE HIGHEST INDIVIDUAL SCORE AMONG THE TWO SFT MODELS.

Model	Optimal hyperparameters	CMMLU	C-Eval	MBPP	GSM8K	GaoKaoBench	Average
Qwen3-1.7B							
Base	-	64.97	57.07	50.80	74.00	15.54	52.48
SFT-1	-	65.52*	64.42	53.60	75.51*	17.50*	55.33*
SFT-2	-	65.41	65.36*	54.80*	72.78	15.29	54.73
Weighted Merging	$\lambda = 0.2$	65.40	64.98	54.60	73.56	16.16	54.94
Ties	-	65.42	65.45	54.90	73.54	17.94	55.45
DARE	-	65.39	65.43	55.40	74.54	16.64	55.48
PCB-Merging	-	65.59	65.31	54.90	74.65	17.84	55.66
SVD-Merging	-	65.57	65.29	54.60	74.89	17.68	55.61
EXPO-1	$\alpha = 0.1$	65.72	65.23	54.60	76.69	18.24	56.10
EXPO-2	$\alpha = 0.1$	66.63	66.30	56.00	72.33	14.93	55.24
ExMe	$\beta = 0.1$	66.36	66.72	55.80	75.84	21.54	57.25
Qwen3-4B							
Base	-	76.51	73.45	67.00	84.61	24.51	64.42
SFT-1	-	76.27*	78.53*	67.6	85.29	21.52	65.84
SFT-2	-	76.05	78.14	68.00*	85.37*	22.30*	65.97*
Weighted Merging	$\lambda = 0.3$	76.15	78.42	67.80	85.28	21.75	65.88
Ties	-	76.32	79.32	68.40	85.45	22.13	66.32
DARE	-	76.39	78.84	68.40	85.52	22.15	66.26
PCB-Merging	-	76.54	78.94	69.40	85.54	22.81	66.65
SVD-Merging	-	76.52	78.68	69.20	85.39	22.48	66.45
EXPO-1	$\alpha = 0.4$	75.94	81.89	70.00	85.37	21.43	66.93
EXPO-2	$\alpha = 0.2$	76.33	81.03	70.20	85.60	21.89	67.01
ExMe	$\beta = 0.2$	76.86	82.22	70.80	85.75	23.40	67.81
Qwen3-8B							
Base	-	80.04	82.51	69.40	82.64	33.58	69.63
SFT-1	-	80.58*	82.66	70.80*	87.72*	31.54*	70.66*
SFT-2	-	80.35	82.90*	70.80	86.66	29.09	69.96
Weighted Merging	$\lambda = 0.1$	80.46	82.80	69.80	86.84	30.67	70.11
Ties	-	80.52	83.32	71.20	86.82	30.54	70.48
DARE	-	80.48	83.12	70.90	86.84	30.62	70.39
PCB-Merging	-	81.24	84.24	70.80	88.32	30.23	70.97
SVD-Merging	-	81.16	84.12	70.60	88.16	30.12	70.83
EXPO-1	$\alpha = 0.2$	81.48	84.75	72.00	89.04	28.63	71.18
EXPO-2	$\alpha = 0.1$	81.14	84.68	71.20	89.81	28.04	70.97
ExMe	$\beta = 0.3$	82.45	84.87	72.80	89.81	28.28	71.64

Solve the following problem **step by step**.
 The last line of your response should be of the form: [boxed{ \$Answer \$ }]
 where \$Answer\$ is the answer to the problem.
Problem: {Problem}

Fig. 3. Prompt for distillation data.

C. Datasets

In the general domain, we adopt the Alpaca dataset, which includes both Chinese and English corpora [37] and consists of 52,000 unique entries. For the mathematical domain, we use DeepMath-103 K as the seed dataset, extract a subset with difficulty scores above 6, and then apply data distillation with DeepSeek-V3 to construct the mathematical fine-tuning dataset. The prompt is shown in Fig. 3.

D. Evaluation

We evaluate the capabilities of the LLMs using OpenCompass [38], an integrated evaluation framework that includes various mainstream evaluation datasets. To conduct a more

comprehensive evaluation of the capabilities of these models, we select the following seven benchmark datasets:

- *GSM8K* [39]: A dataset of 8.5 K high-quality elementary school math word problems created by human writers.
- *MATH-500* [40]: A subset of the MATH dataset, consisting of 500 selected problems for evaluation.
- *HumanEval* [41]: A dataset for evaluating code generation models containing 164 programming problems, each including function signatures, docstrings, function bodies, and multiple unit tests.
- *CMMLU* [42]: A Chinese multiple-choice dataset evaluating language understanding and knowledge mastery.
- *C-Eval* [43]: A authoritative Chinese evaluation dataset containing 13,948 multiple-choice questions across 52 disciplines at four difficulty levels (middle school, high school, university, and professional).
- *GaoKaoBench* [44]: A benchmark containing questions from China's national college entrance exams (Gaokao).
- *AIME24* [45]: A dataset of 30 highly challenging mathematical problems designed for top-tier high school students, used to evaluate model performance on advanced mathematical reasoning and problem-solving.

TABLE II

THIS TABLE SHOWS THE SCORES OF QWEN3-1.7B, QWEN3-4B, AND QWEN3-8B USING DIFFERENT METHODS, WITH SFT MODELS TRAINED ON THE MATH DATASET. * INDICATES THE HIGHEST INDIVIDUAL SCORE AMONG THE TWO SFT MODELS.

Model	Optimal hyperparameters	MATH-500	GSM8K	AIME24	Average	Average (excl. MATH)
Qwen3-1.7B						
Base	-	46.20	74.00	6.67	42.29	40.34
SFT-1	-	55.20	77.49	10.00*	47.56*	43.75*
SFT-2	-	57.20*	77.91*	6.67	47.26	42.29
Weighted Merging	$\lambda = 0.1$	56.40	76.99	13.33	48.91	45.16
Ties	-	57.80	77.62	13.33	49.58	45.48
DARE	-	58.60	78.98	10.00	49.19	44.49
PCB-Merging	-	57.60	78.19	10.00	48.60	44.10
SVD-Merging	-	59.20	79.39	13.33	50.64	46.36
EXPO-1	$\alpha = 0.2$	58.20	79.59	13.33	50.37	46.46
EXPO-2	$\alpha = 0.1$	59.00	78.56	10.00	49.19	44.28
ExMe	$\beta = 0.2$	59.80	80.98	13.33	51.37	47.16
Qwen3-4B						
Base	-	54.10	84.61	6.67	48.46	45.64
SFT-1	-	67.00	85.96	10.00*	54.32	47.98
SFT-2	-	69.80*	86.24*	10.00*	55.35*	48.12*
Weighted Merging	$\lambda = 0.3$	69.80	86.68	13.33	56.60	50.01
Ties	-	69.20	87.06	10.00	55.42	48.53
DARE	-	67.80	85.11	13.33	55.41	49.22
PCB-Merging	-	71.60	85.71	13.33	56.88	49.52
SVD-Merging	-	70.80	85.19	16.67	57.55	50.93
EXPO-1	$\alpha = 0.2$	70.20	87.07	10.00	55.76	48.54
EXPO-2	$\alpha = 0.1$	72.00	86.99	13.33	57.44	50.16
ExMe	$\beta = 0.1$	72.60	88.06	16.67	59.11	52.37
Qwen3-8B						
Base	-	64.40	82.64	16.67	54.57	49.66
SFT-1	-	69.60	85.78	23.33*	59.57	54.56
SFT-2	-	70.40*	86.72*	23.33*	60.15*	55.03*
Weighted Merging	$\lambda = 0.2$	69.00	85.32	26.67	60.33	56.00
Ties	-	73.00	86.38	23.33	60.90	54.86
DARE	-	73.60	89.98	26.67	63.42	58.33
PCB-Merging	-	72.40	90.00	23.33	61.91	56.67
SVD-Merging	-	74.80	91.64	23.33	63.26	57.49
EXPO-1	$\alpha = 0.1$	75.40	90.66	26.67	64.24	58.67
EXPO-2	$\alpha = 0.2$	74.40	91.25	23.33	62.99	57.29
ExMe	$\beta = 0.2$	76.80	91.05	26.67	64.84	58.86

E. Detail

We train the base versions of Qwen3-1.7B, Qwen3-4B, and Qwen3-8B, which have not undergone instruction fine-tuning, using the datasets described in IV-C. Since these models have not yet learned to follow instructions, we use the Alpaca dataset to enhance their instruction-following capabilities, thereby improving their performance across various tasks. Specifically for mathematical tasks, using a purely mathematical dataset effectively enhances the models' reasoning ability in this domain. When selecting candidate SFT and EXPO models, we adopt different strategies to validate the effectiveness of our method. For general tasks, we use the average performance across all tasks as the selection criterion. For mathematical tasks, we use only the MATH-500 benchmark.

V. RESULTS AND ANALYSIS

A. Overview

We conduct a detailed comparison of performance improvements achieved by different model merging methods across various Qwen3 series base models, as shown in Tables I and II.

Specifically, Table I corresponds to using the Alpaca dataset to enhance the instruction-following capabilities of the base models, while Table II corresponds to using mathematically related datasets to improve the mathematical reasoning abilities of the base models. These tables present evaluation scores for three Qwen3 series base models and their variants after applying SFT, Weighted Merging, TIES-Merging, DARE-Merging, PCB-Merging, SVD-Merging, Model Extrapolation, and Extrapolation Merging across multiple benchmarks.

The merging hyperparameters for both Weighted Merging and ExMe are tuned within 0.1–0.9, while those for standalone Model Extrapolation are set to 0.1–0.5. For models with identical architectures, vanilla Weighted Merging cannot consistently yield performance gains: under the overall task setting in Table I, its optimal merged performance falls between the two original SFT models, whereas under the mathematical task setting in Table II, the best merged results exceed both individual SFT counterparts. We attribute this discrepancy to the distinct training objectives of the two experimental setups. In addition, conventional Weighted Merging only delivers marginal performance gains, with mere improvements of 1.41, 1.89, and 0.97 points on the math-trained Qwen3-1.7B, Qwen3-4B,

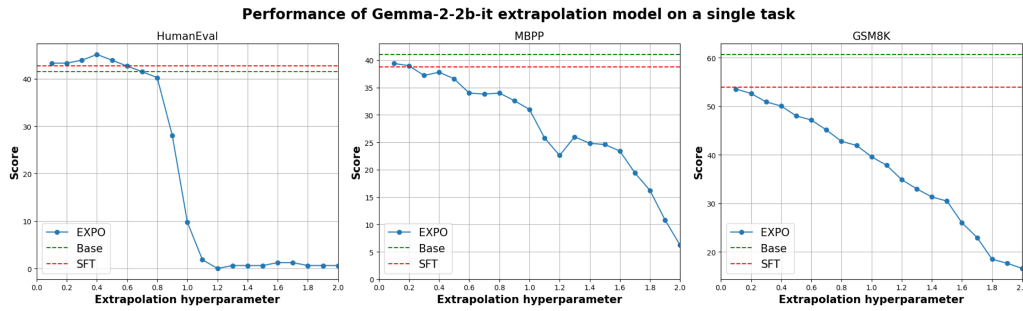


Fig. 4. The performance of the extrapolated gemma-2-2b-it model on HumanEval, MBPP, and GSM8K.

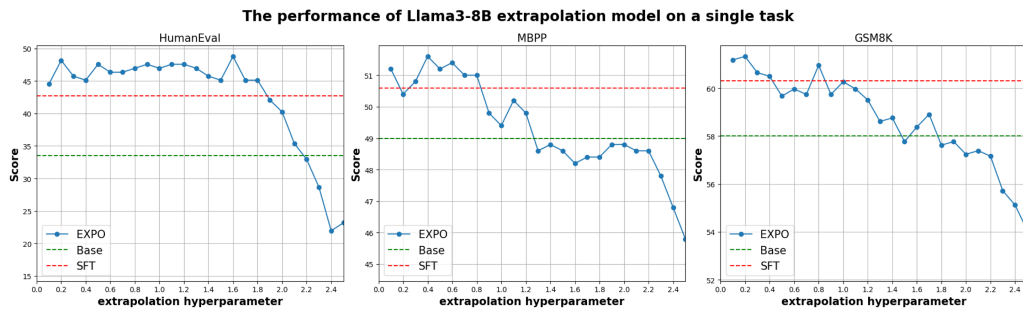


Fig. 5. The performance of the extrapolated Llama3-8B model on HumanEval, MBPP, and GSM8K.

and Qwen3-8B respectively, which clearly reveals the inherent limitations of traditional merging strategies. By contrast, Model Extrapolation and ExMe can consistently locate proper hyperparameter configurations to make the final model outperform the original baselines, demonstrating that EXPO and ExMe achieve more stable and effective performance improvements than vanilla Weighted Merging.

Among all compared methods, *ExMe* achieves the greatest improvement in model performance. Specifically, compared to the best SFT model, on instruction-following tasks, *ExMe* improves the average score of Qwen3-1.7B by 1.92 points, Qwen3-4B by 1.84 points, and Qwen3-8B by 0.98 points. On individual benchmarks, *ExMe* also achieves the best results: on C-Eval, the three models improve by 1.36, 3.69, and 1.97 points, respectively; on GSM8K, the improvements are 0.33, 0.38, and 2.09 points, respectively. These results demonstrate that *ExMe* exhibits strong generalization ability and stability on instruction-following tasks. On mathematical tasks, the performance improvements are even more significant. Compared to the best SFT model, *ExMe* improves the average score of Qwen3-1.7B by 3.41 points, Qwen3-4B by 4.25 points, and Qwen3-8B by 3.83 points. On the challenging mathematical reasoning benchmark AIME24, *ExMe* achieves the best individual scores across all three model scales: 13.33 for Qwen3-1.7B, 16.67 for Qwen3-4B, and 26.67 for Qwen3-8B, surpassing existing SFT models and other model merging methods. Overall, *ExMe* outperforms existing methods across different tasks and model scales, demonstrating good scalability.

Depending on the selection criterion, the improvement objective pursued by *ExMe* also differs. From the experimental results, for instruction-following tasks, we adopt average

performance as the criterion to select both SFT models and EXPO models. Under this strategy, Extrapolation Merging primarily enhances the model's overall performance but does not achieve consistent improvements across all individual capabilities, especially those that deteriorate after SFT. For example, after SFT, Qwen3-4B drops by 2.21 points on GaoKaoBench, and Qwen3-8B drops by 2.04 points. Extrapolation Merging partially recovers this decline on Qwen3-4B, improving by 1.10 points, but leads to further degradation on Qwen3-8B. For mathematical tasks, we instead adopt MATH-500 as a single criterion to select both SFT models and EXPO models. In this case, Extrapolation Merging primarily enhances the model's mathematical capabilities. Specifically, across all three model sizes, *ExMe* achieves the best overall and per-task mathematical performance. The only exception is on GSM8K with Qwen3-8B, where *ExMe* slightly underperforms SVD-Merging but still exceeds both SFT models. We attribute this difference to the distinct training objectives of the two task types: instruction-following tasks aim to teach the base model to follow instructions and respond accordingly, so the evaluation results across different tasks may rise or fall without a unified direction of optimization. In contrast, mathematical tasks have a highly consistent optimization direction, and when evaluated solely on mathematically relevant benchmarks, more significant performance improvements can be observed.

B. Extrapolation Stage Analysis

We respectively verify the usefulness of the model extrapolation method during the SFT stage from the perspectives of

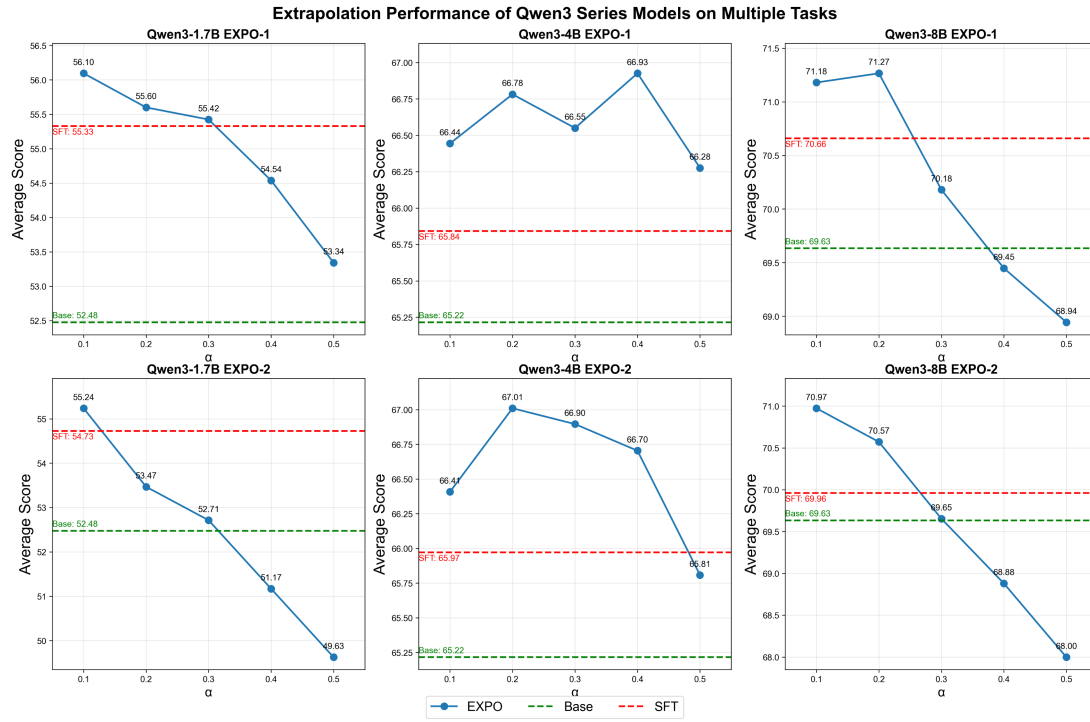


Fig. 6. This figure shows the impact of the extrapolation hyperparameter α on the performance of the extrapolated models across three different models, where each of the six selected SFT models is extrapolated from the base model. The blue line represents the performance of the extrapolated models, while the red dashed line indicates the performance of the SFT models involved in the extrapolation.

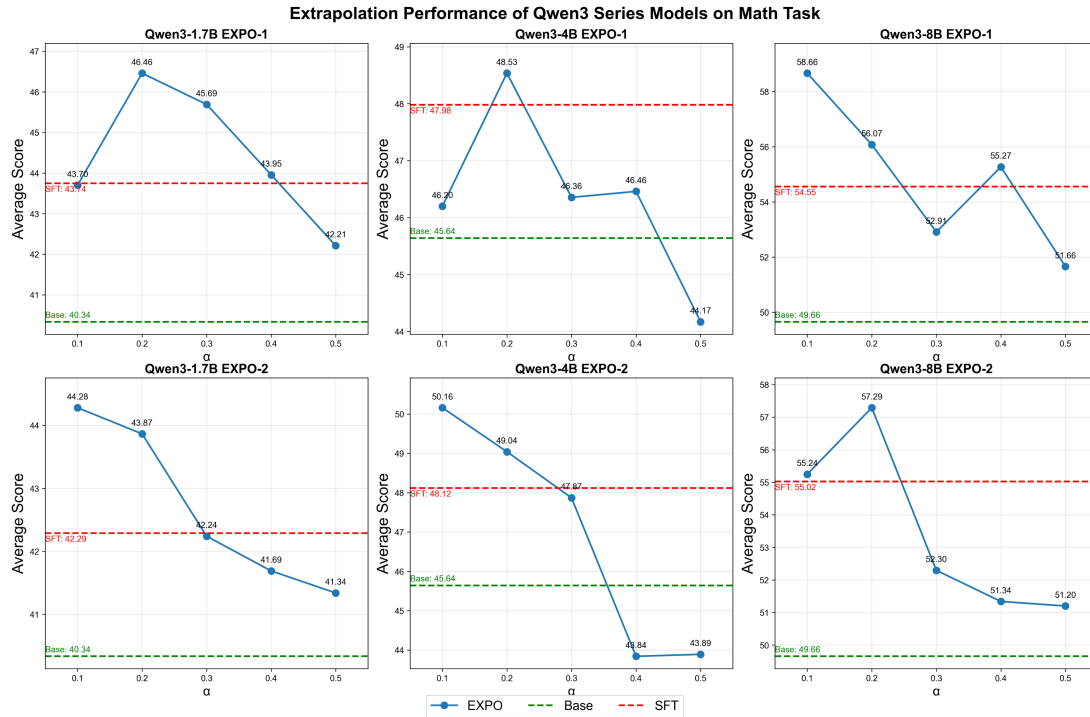


Fig. 7. This figure shows the impact of the extrapolation hyperparameter α on the performance of the extrapolated models across three different models, where each of the six selected SFT models is extrapolated from the base model. The blue line represents the performance of the extrapolated models, while the red dashed line indicates the performance of the SFT models involved in the extrapolation.

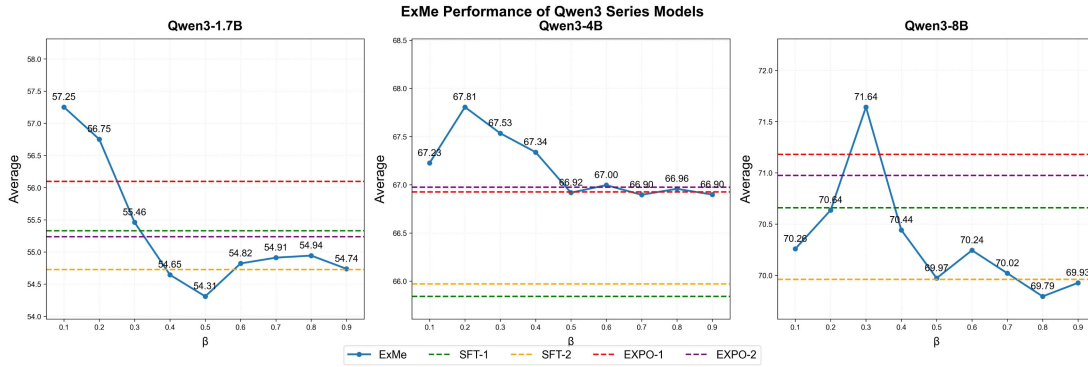


Fig. 8. This figure illustrates the impact of different merging hyperparameters on overall performance when training on the Alpaca dataset and using ExMe.

single performance and comprehensive performance. Specifically, we select checkpoints of fine-tuned models from the fine-tuning stage as strong models and use the base model as the weak model to perform model extrapolation. In terms of single performance, we employ gemma-2-2b-it [46] and Llama3-8B [47], two models from different families than Qwen3, and extend the extrapolation hyperparameter to 2 to validate generalizability. In terms of comprehensive performance, we continue to conduct experiments on the Qwen3 series of models.

Model extrapolation is affected by parameter changes during the fine-tuning phase and has performance limits. As shown in Fig. 4 and Fig. 5 when the SFT model outperforms the base model, as the extrapolation coefficient increases, the performance of the extrapolated model first improves and then declines. Compared with the SFT model, the extrapolated model of gemma-2-2b-it achieves a maximum improvement of 2.44 points on HumanEval; the extrapolated model of Llama3-8B achieves a maximum improvement of 5.49 points on HumanEval, 1.00 point on MBPP, and 1.0 point on GSM8K. This indicates that the extrapolation method has an upper bound in improving model performance. When the extrapolation coefficient expands to a certain extent, model performance deteriorates catastrophically, sometimes even falling below the performance of the base model. When the SFT model underperforms relative to the base model, as shown in Fig. 4, the performance of the extrapolated model of gemma-2-2b-it on MBPP and GSM8K consistently declines as the extrapolation coefficient increases, and falls below the performance of both models involved in extrapolation. In summary, the extrapolation method can amplify the impact of fine-tuning on model performance to some extent.

When considering overall performance, the model extrapolation method remains effective. As shown in Fig. 6 and Fig. 7, the experimental results based on the Qwen3 series of models demonstrate that we can always find suitable extrapolation hyperparameters such that the comprehensive performance of the extrapolated model outperforms the original SFT model. In some cases, all variants generated after extrapolation even achieve comprehensive performance superior to that of the original SFT model. This result confirms the effectiveness of model extrapolation with base and SFT models.

Extrapolation hyperparameters significantly impact the performance of extrapolated models. Furthermore, when extrapolating between the base model and the SFT model—regardless

of whether the model is trained on instruction-following tasks or mathematical tasks—Qwen3-1.7B, Qwen3-4B, and Qwen3-8B exhibit in most cases a general trend where the overall performance of the extrapolated model fluctuates and declines as the extrapolation hyperparameter increases, sometimes even monotonically decreasing. This phenomenon arises because the performance gain brought by fine-tuning is limited, and the direction of parameter optimization is not sufficiently clear; we can only achieve modest improvements near a local optimum. Although small-scale extrapolation around this local point may potentially move closer to a better solution, model extrapolation fundamentally amplifies the performance changes induced by fine-tuning. When the direction is inaccurate, this amplification instead exacerbates instability, leading to performance fluctuations or decline. Therefore, in such cases, only a small extrapolation coefficient can be adopted to mitigate performance loss. This demonstrates that the extrapolation hyperparameter significantly influences the performance of the extrapolated model, and its selection is highly dependent on the specific task and model, with no universal value applicable across all scenarios.

C. Merging Stage Analysis

In this section, we illustrate the effectiveness of ExMe by analyzing the experimental results from the merging stage. Additionally, we examine the impact of the merging hyperparameter β on the overall performance of all candidate models during the merging stage of ExMe, as illustrated in Figs. 8 and 9.

Most candidate models outperform the SFT models involved in the merging stage. Specifically, when training on the Alpaca dataset, three candidate models of Qwen3-1.7B outperform the SFT model in terms of overall performance; all candidate models of Qwen3-4B surpass the SFT model; and for Qwen3-8B, only one candidate model exceeds the SFT model in overall performance. When training on the mathematical dataset, except for one candidate model of Qwen3-1.7B that underperforms the best SFT model, all other candidate models outperform the best SFT model.

Model obtained by merging with a smaller β value typically achieves the best overall performance. From Fig. 8 and Fig. 9, we observe that when training on the Alpaca dataset, the optimal β values for Qwen3-1.7B, Qwen3-4B, and Qwen3-8B are 0.1, 0.2, and 0.3, respectively. When training on the mathematical

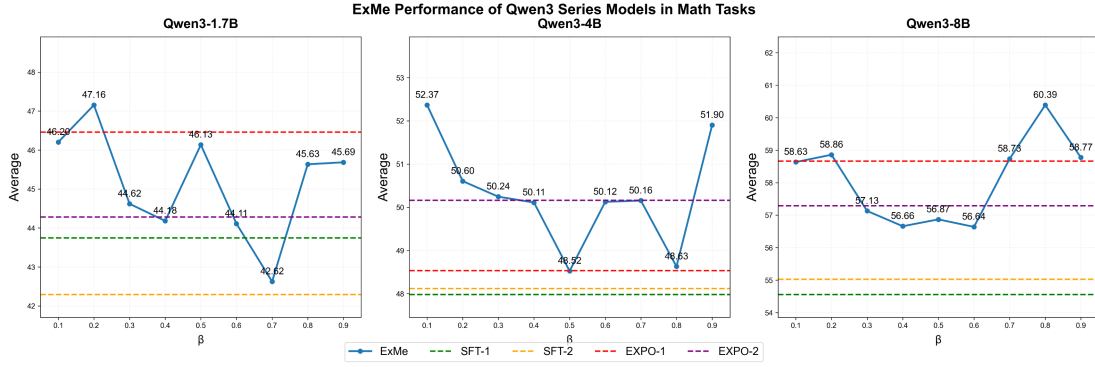


Fig. 9. This figure illustrates the impact of different merging hyperparameters on overall performance (excluding MATH-500) when training on a math dataset and using ExMe.

dataset, the optimal β values for the three models are 0.2, 0.1, and 0.8, respectively. This indicates a clear primary-secondary relationship between the two extrapolated models during the merging process. We attribute this to the use of the weaker extrapolated model to correct the parameters of the stronger extrapolated model, thereby achieving better performance.

VI. CONCLUSION

In this work, we validate the effectiveness of applying Model Extrapolation between the base model and the SFT model, and reveal that its performance is inherently limited by the fixed direction of linear extrapolation.

To overcome this performance ceiling, we further propose Extrapolation Merging (ExMe). This method introduces Weighted Merging after Model Extrapolation to mitigate the performance degradation caused by the rigid linear optimization direction, thereby consistently improving the overall performance of the fine-tuned model without requiring any additional training.

Experimental results show that, compared to conventional Weighted Merging and Model Extrapolation approaches, ExMe achieves not only significantly better overall performance but also greater robustness.

REFERENCES

- [1] W. X. Zhao et al., "A survey of large language models," *Front. Comput. Sci.*, vol. 20, 2026, Art. no. 2012627.
- [2] Hidayaturrahman and I. Prawira, "Retrieval-augmented generation for social media content creation with sentence window and auto-merging retrieval," in *Proc. Int. Conf. Informat., Multimedia, Cyber Inf. Syst.*, 2024, pp. 608–613.
- [3] D. Chiba, H. Nakano, and T. Koide, "DomainLynx: Advancing LLM techniques for robust domain squatting detection," *IEEE Access*, vol. 13, pp. 29914–29931, 2025.
- [4] H. Nakanishi, K. Hasegawa, S. Hidano, K. Fukushima, K. Hashimoto, and N. Togawa, "Performance comparison of the LLM models on LLM-based seed generation method for IoT device fuzzing," in *Proc. IEEE Int. Conf. Consum. Electron.*, 2025, pp. 1–5.
- [5] J. Wei et al., "Finetuned language models are zero-shot learners," in *Proc. 10th Int. Conf. Learn. Representations*, 2022.
- [6] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst. 35: Annu. Conf. Neural Inf. Process. Syst.*, New Orleans, LA, USA, 2022, pp. 27730–27744.
- [7] J. Hoffmann et al., "Training compute-optimal large language models," *Adv. Neural Inf. Process. Syst. 35*, pp. 30016–30030, 2022.
- [8] Y. Pinter and M. Elhadad, "Emptying the ocean with a spoon: Should we edit models?," in *Proc. Findings Assoc. Comput. Linguistics*, Singapore, Association for Computational Linguistics, 2023, pp. 15164–15 172.
- [9] P. Wang et al., "EasyEdit: An easy-to-use knowledge editing framework for large language models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, Aug. 2024, pp. 82–93.
- [10] Y. Yao et al., "Editing large language models: Problems, methods, and opportunities," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Singapore Association for Computational Linguistics, 2023, pp. 10222–10240.
- [11] J. Wang et al., "The missing piece in model editing: A deep dive into the hidden damage brought by model editing," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2025, pp. 1–5.
- [12] H. Zhang, F. Stahlberg, and S. Kumar, "Predicting compact phrasal rewrites with large language models for ASR post editing," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2025, pp. 1–5.
- [13] Y. Li, Y. Gong, C.-H. H. Yang, P. Bell, and C. Lai, "Revise, reason, and recognize: LLM-based emotion recognition via emotion-specific prompts and asr error correction," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2025, pp. 1–5.
- [14] X. Jin, X. Ren, D. Preotiuc-Pietro, and P. Cheng, "Dataless knowledge fusion by merging weights of language models," in *Proc. 11th Int. Conf. Learn. Representations*, Kigali, Rwanda, 2023.
- [15] G. Ilharco, M. T. Ribeiro, M. Wortsman, L. Schmidt, H. Hajishirzi, and A. Farhadi, "Editing models with task arithmetic," in *Proc. 11th Int. Conf. Learn. Representations*, Kigali, Rwanda, 2023.
- [16] P. Yadav, D. Tam, L. Choshen, C.A. Raffel, and M. Bansal, "TIES-Merging: Resolving interference when merging models," in *Proc. Adv. Neural Inf. Process. Syst. 36: Annu. Conf. Neural Inf. Process. Syst.*, New Orleans, LA, USA, 2023, pp. 7093–7115.
- [17] L. Yu, B. Yu, H. Yu, F. Huang, and Y. Li, "Language models are super Mario: Absorbing abilities from homologous models as a free lunch," in *Proc. 41st Int. Conf. Mach. Learn.*, PMLR, vol. 235, 2024, pp. 57755–57775.
- [18] Y. Lin et al., "Mitigating the alignment tax of RLHF," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 580–606.
- [19] C. Zheng, Z. Wang, H. Ji, M. Huang, and N. Peng, "Weak-to-strong extrapolation expedites alignment," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, Vienna, Austria Jul. 2025, pp. 1025–1041.
- [20] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. Adv. Neural Inf. Process. Syst. 36: Annu. Conf. Neural Inf. Process. Syst.*, New Orleans, LA, USA, 2023, pp. 53728–53741.
- [21] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process.*, Association for Computational Linguistics, 2021, pp. 4582–4597.
- [22] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proc. Conf. Empirical Methods Natural Lang. Proc.*, Association for Computational Linguistics, 2021, pp. 3045–3059.
- [23] E. J. Hu et al., "LoRa: Low-rank adaptation of large language models," in *Proc. 10th Int. Conf. Learn. Representations*, 2022.

- [24] M. Wortsman et al., "Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," in *Proc. Int. Conf. Mach. Learn.*, Baltimore, Maryland, USA, 2022, pp. 23965–23998.
- [25] M. Matena and C. Raffel, "Merging models with fisher-weighted averaging," in *Proc. Adv. Neural Inf. Process. Syst. 35: Annu. Conf. Neural Inf. Process. Syst.*, New Orleans, LA, USA, 2022, pp. 17703–17716.
- [26] D. Liu et al., "Maximizing intermediate checkpoint value in LLM pretraining with Bayesian optimization," in *Proc. 42nd Int. Conf. Mach. Learn.*, PMLR, vol. 267, 2025, pp. 39713–39741.
- [27] X. Wu, S. Huang, and F. Wei, "Mixture of LoRa experts," in *Proc. 12th Int. Conf. Learn. Representations*, Vienna, Austria, 2024.
- [28] G. Du et al., "Parameter competition balancing for model merging," in *Proc. Adv. Neural Inf. Process. Syst. 38: Annu. Conf. Neural Inf. Process. Syst.*, Vancouver, BC, Canada, 2024, pp. 84746–84776.
- [29] G. Stoica, P. Ramesh, B. Ecsedi, L. Choshen, and J. Hoffman, "Model merging with SVD to tie the knots," in *Proc. 13th Int. Conf. Learn. Representations*, Singapore, 2025.
- [30] F. Draxler, K. Veschgini, M. Salmhofer, and F. Hamprecht, "Essentially no barriers in neural network energy landscape," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1309–1318.
- [31] R. Entezari, H. Sedghi, O. Saukh, and B. Neyshabur, "The role of permutation invariance in linear mode connectivity of neural networks," 2021, *arXiv:2110.06296*.
- [32] T. Garipov, P. Izmailov, D. Podoprikin, D. P. Vetrov, and A. G. Wilson, "Loss surfaces, mode connectivity, and fast ensembling of DNNs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018.
- [33] G. Ortiz-Jimenez, A. Favero, and P. Frossard, "Task arithmetic in the tangent space: Improved editing of pre-trained models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 66727–66754.
- [34] R. Jin, B. Hou, J. Xiao, W. Su, and L. Shen, "Fine-tuning attention modules only: Enhancing weight disentanglement in task arithmetic," 2024, *arXiv:2407.07089*.
- [35] V. Nagarajan and J. Z. Kolter, "Uniform convergence may be unable to explain generalization in deep learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 11615–11626.
- [36] A. Yang et al., "Qwen3 technical report," 2025, *arXiv:2505.09388*.
- [37] B. Peng, C. Li, P. He, M. Galley, and J. Gao, "Instruction tuning with GPT-4," 2023.
- [38] O. Contributors, "OpenCompass: A universal evaluation platform for foundation models," 2023. [Online]. Available: <https://github.com/opencompass/opencompass>
- [39] K. Cobbe et al., "Training verifiers to solve math word problems." 2021, *arXiv:2110.14168*.
- [40] D. Hendrycks et al., "Measuring mathematical problem solving with the MATH dataset," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks 1*, 2021.
- [41] M. Chen et al., "Evaluating large language models trained on code." 2021, *arXiv:2107.03374*.
- [42] H. Li et al., "CMMLU: Measuring massive multitask language understanding in Chinese," *Findings Assoc. Comput. Linguistics*, Bangkok, Thailand, pp. 11260–11285, Aug. 2024.
- [43] Y. Huang et al., "C-EVAL: A multi-level multi-discipline chinese evaluation suite for foundation models," in *Proc. Adv. Neural Inf. Process. Syst. 36: Annu. Conf. Neural Inf. Process. Syst.*, New Orleans, LA, USA, 2023, pp. 62991–63010.
- [44] X. Zhang, C. Li, Y. Zong, Z. Ying, L. He, and X. Qiu, "Evaluating the performance of large language models on GAOKAO benchmark," 2023, *arXiv:2305.12474*.
- [45] Y. Zhang and Math-AI Team, "American invitational mathematics examination (AIME) 2024," 2024. [Online]. Available: https://huggingface.co/datasets/Maxwell-Jia/AIME_2024
- [46] G. Team et al., "Gemma 2: Improving open language models at a practical size," 2024, *arXiv:2408.00118*.
- [47] A. Grattafiori et al., "The Llama 3 herd of models," 2024, *arXiv:2407.21783*.